

SpringBoot 3.5.0 + JPA + JSP + IntelliJ IDE

회원관리 + 게시판 + 댓글 실습

1. 주요기능

- 1) 회원관리(로그인 /로그아웃) - Users Table
- 2) 자유게시판(등록, 수정, 삭제, 상세보기 ,전체검색) - FreeBoard Table
- 3) 댓글(등록, 검색) - Reply Table

2. 주요기술 및 환경

- 1) Spring WEB MVC구조
- 2) Spring AOP - Session유무체크
: FreeBoard에 대한 접근은 인증된 사용자만 가능
- 3) Spring Data JPA
- 4) @ControllerAdvice - 예외처리
- 5) Action Tag include - Layout
- 6) Bootstrap UI
- 7) MySQL
- 8) 게시판에 paging처리

3. 프로젝트 생성시 필요한 LIB

Web / jpa / mySQL / Lombok / dev tools /

*추가적인 dependency는 프로젝트 생성후에 추가한다.

- : JSP 설정
- : QueryDSL 설정

* application.properties파일 수정

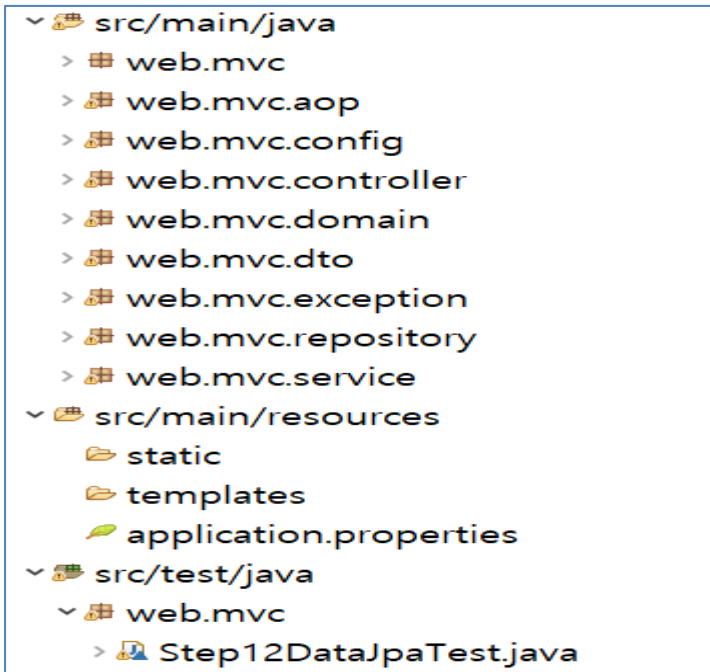
```
1  spring.application.name=step02_UserBoardReply
2  #port
3  server.port=9000
4
5  logging.level.root=info
6
7  logging.level.org.hibernate.SQL=debug
8  logging.level.org.springframework.orm.jpa.JpaTransactionManager=debug
9  logging.level.org.springframework.transaction.interceptor.TransactionInterceptor=trace
10
11  #jsp view
12  spring.mvc.view.prefix=/WEB-INF/views/
13  spring.mvc.view.suffix=.jsp
14
15  spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
16  spring.datasource.url=jdbc:mysql://localhost:3307/jpa?serverTimezone=Asia/Seoul
17  spring.datasource.username=jang
18  spring.datasource.password=jang1234
19
20  spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect
21  spring.jpa.hibernate.ddl-auto=none
22  spring.jpa.generate-ddl=false
23  spring.jpa.show-sql=true
24  spring.jpa.database=mysql
25
```

* View template jsp를 위한 설정

src/main/webapp/WEB-INF/views폴더 생성

4. 폴더구조

1) back End 폴더



```

  ▾ web.mvc.repository
    > FreeBoardRepository.java
    > ReplyRepository.java
    > UserRepository.java
  ▾ web.mvc.service
    > FreeBoardService.java
    > FreeBoardServiceImpl.java
    > ReplyService.java
    > ReplyServiceImpl.java
    > UserService.java
    > UserServiceImpl.java

```

2) Front End 구조

```

  ▾ webapp
    ▾ css
      bootstrap.min.css
      style.css
    > js
    ▾ WEB-INF
      ▾ views
        > board
        > common
        > error
        > reply
        > user
        index.jsp

```

API설계(API명세서_USER_Board_Reply.xlsx 문서 참고)

번호	Controller	기능	요청방식	URI
1	HomeController	메인화면 요청	get	/
2	UserController	로그인폼	get	/user/{url}
3		로그인	post	/user/loginCheck
4		로그아웃	get	/user/logout
5	FreeBoardController	폼 화면 요청	get	/board/{url}
6		전체목록검색	get	/board/list
7		게시물 등록	post	/board/insert
8		상세보기	get	/board/read/{bno}
9		수정하기 폼	post	/board/updateForm
10		수정하기	post	/board/update
11		삭제하기	post	/board/delete
12	ReplyController	댓글등록폼	post	/reply/writeForm
13		댓글등록	post	/reply/insert
14		댓글삭제	get	/reply/delete/{rno}/{bno}

실행화면

1) <http://localhost:9000/>

← → ↻ ⓘ localhost:9000

Home Login Board

My Spring Boot + JPA Final(USER and Board and Reply)!!!!

Spring Final Wow!

1. 주요기능
 - 1) 회원관리(로그인 / 로그아웃) - Users Table
 - 2) 자유게시판(등록, 수정, 삭제, 상세보기, 전체검색) - FreeBoard Table
 - 3) 댓글(등록, 검색) - Reply Table
2. 주요 기술 및 환경
 - 1) Spring WEB MVC구조
 - 2) Spring AOP - Session유무 체크
: FreeBoard에 대한 접근은 인증된 사용자만 가능
 - 3) Spring Data JPA
 - 4) @ControllerAdvice - 예외처리
 - 5) Action Tag include - Layout Template
 - 6) Bootscrap UI
 - 7) Oracle 21c
 - 8) 게시판에 paging처리

Wow!

Spring MVC 6/ JPA / AOP - 장희정 Programmer.

메뉴에서 로그인 클릭

← → ↻ ⓘ localhost:9000/user/login

Home Login Board

Login

Login, AOP/HttpSession

[Learn more](#)

Login

User Id	userid
Password	password
	Cancel Submit

Wow!

Spring MVC 6/ JPA / AOP - 장희정 Programmer.

로그인 성공 후

로그인성공 후 header

My Spring Boot + JPA Final(USER and Board and Reply)!!!!

Spring Final Wow!

1. 주요기능
 - 1) 회원관리(로그인 / 로그아웃) - Users Table
 - 2) 자유게시판(등록, 수정, 삭제, 상세보기, 전체검색) - FreeBoard Table
 - 3) 댓글(등록, 검색) - Reply Table
2. 주요 기술 및 환경
 - 1) Spring WEB MVC구조
 - 2) Spring AOP - Session유무 체크
: FreeBoard에 대한 접근은 인증된 사용자만 가능
 - 3) Spring Data JPA
 - 4) @ControllerAdvice - 예외처리
 - 5) Action Tag include - Layout Template
 - 6) Bootscrap UI
 - 7) Oracle 21c
 - 8) 게시판에 paging처리

Wow!

Spring MVC 6/ JPA / AOP - 장희정 Programmer.

전체게시물 조회

<http://localhost:9000/board/list>

자유게시판

번호	제목	작성자	날짜	조회수
163	제목53 / Reply count : 0	User53	2024-05-11T15:02:59.347030	2
162	제목52 / Reply count : 0	User52	2024-05-11T15:02:59.346032	0
161	제목51 / Reply count : 0	User51	2024-05-11T15:02:59.345033	0
160	제목50 / Reply count : 0	User50	2024-05-11T15:02:59.345033	0
159	제목49 / Reply count : 0	User49	2024-05-11T15:02:59.344036	0

제목클릭

<http://localhost:9000/board/read/163>

89 게시물 수정하기

수정하기 다시쓰기

페이지 8 / 11

localhost:9000/reply/writeForm

Home Login Board web.mvc.domain.User@1ca60544! Logout

90부모글에 Reply 등록

내용	
----	--

글쓰기 | 다시쓰기

<리스토로 돌아가기>

Wow!
Spring MVC 6/ JPA / AOP - 장희정 Programmer.

댓글 등록 후

<http://localhost:9000/board/read/90?flag>

localhost:9000/board/read/90?flag

Home Login Board web.mvc.domain.User@1ca60544! Logout

게시물 자세히보기

번호	90
등록일	2024-10-31T20:00:43.205723
조회수	2
제목	제목 45
설명내용	45

수정하기 삭제하기 답변하기

Reply INFO

1순서 / 1 = 첫번째 게시물입니다. / 2024-10-31T20:50:12.132370 삭제
2순서 / 2 = 두번째 게시물입니다. / 2024-10-31T20:50:21.134062 삭제
3순서 / 3 = 세번째 게시물입니다. / 2024-10-31T20:50:29.264666 삭제

<리스토로 돌아가기>

Wow!
Spring MVC 6/ JPA / AOP - 장희정 Programmer.

예외처리

: 각 상황에 따른 예외를 적절하게 처리한다.

```

public enum ErrorCode { //enum은 'Enumeration' 의 약자로 열거, 목록 이라는 뜻

    ACCESS_DENIED( status: 600, msg: "로그인하고 이용해주세요."),
    NOTFOUND_ID( status: 601, msg: "존재하지 않는 ID입니다."),
    WRONG_PASS( status: 602, msg: "비밀번호 오류입니다.."),

    FAILED_DETAIL( status: 603, msg: "상세보기 오류입니다."),
    FAILED_UPDATE( status: 604, msg: "글번호 오류로 수정할수 없습니다."),
    FAILED_DELETE( status: 605, msg: "글을 삭제할수 없습니다.^^.");

    private final int status;
    private final String msg;
}

```

```

14 usages new *
@Getter
@RequiredArgsConstructor
public class BasicException extends RuntimeException {
    private final ErrorCode errorCode;
}

```

```

new *
@ControllerAdvice
@Slf4j
public class GlobalExceptionHandler {
    new *
    @ExceptionHandler(BasicException.class)
    public ModelAndView error(BasicException e) {
        log.error("Error Message error {} " , e.getErrorCode().getMsg() );

        // 예외가 발생했을때 해야할 일작성
        ModelAndView mv = new ModelAndView( viewName: "error/errorView");
        mv.addObject( attributeName: "errMessage", e.getErrorCode().getMsg());
        mv.addObject( attributeName: "errStatus", e.getErrorCode().getStatus());

        return mv;
    }
}

```

제공된 자료를 바탕으로 세팅을 완료 한 후

XxxController , XxxServiceImpl , XxxRepository를 만들어 전체 프로세스가 동작하도록 한다.

test폴더에서 단위테스트 를 이용하여 user정보와 게시물을 등록한다.

아래 소스를 참고한다.

```
/**
 * 회원등록
 * */
@Test
@Rollback(value = false)
public void userInsert(){
    userRepository.save(User.builder().userId("chan").pwd("1234").name("이찬범").build());
    userRepository.save(User.builder().userId("jang").pwd("1234").name("장희정").build());
    userRepository.save(User.builder().userId("lee").pwd("1234").name("이가현").build());
}
```

```
/**
 * 게시물 샘플등록
 * */
@Test
@Rollback(value = false)
public void boardInsert() {
    for(int i=1; i<=45; i++){
        freeBoardRepository.save(FreeBoard.builder()
            .subject("제목"+i)
            .writer("User"+i)
            .readnum(0)
            .content("내용"+i)
            .password("1234")
            .build());
    }
}
```

♡수고하셨습니다.